A PROJECT REPORT ON

“Design and Development of a Browser History Forensic Analysis Tool”

SUBMITTED BY

Vishwajeet Tulse	**Roll No: B5533**
Pravin Misal	**Roll No: _____**
Rohan Gund	**Roll No: _____**



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE -41

SAVITRIBAI PHULE PUNE UNIVERSITY

2026-2027

Sinhgad Institutes

CERTIFICATE

This is to certify that the project report entitled

“Design and Development of a Browser History Forensic Analysis Tool”

Submitted by

Vishwajeet Tulse	**Roll No: B5533**
Pravin Misal	**Roll No: _____**
Rohan Gund	**Roll No: _____**

Are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. S. U. Mali** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

(Prof. S. U. Mali)	**(Prof. S. P. Bendale)**
Subject In-charge	**Head of Department**

ACKNOWLEDGEMENT

The present world of competition there is a race of existence in which those who have the will to come forward succeed. Project is like a bridge between theoretical and practical work. With this willing we joined this particular project. First of all, we would like to thank the supreme power the Almighty God who is obviously the one who has always guided us to work on the right path of life.

We sincerely thank, Prof. S. P. Bendale sir, Head of the Department of Computer Engineering of NBN Sinhgad School of Engineering, for all the facilities provided to us in the pursuit of this project.

We are indebted to our project guide Prof. S. U. Mali sir, Department of Computer Engineering of NBN Sinhgad School of Engineering. We feel it's a pleasure to be indebted to our guide for his valuable support, advice and encouragement and we thank him for his superb and constant guidance towards this project.

We are deeply grateful to all the staff members of the Computer Engineering department, for supporting us in all aspects.

We acknowledge our deep sense of gratitude to our loving parents for being a constant source of inspiration and motivation.

Vishwajeet Tulse
Pravin Misal
Rohan Gund

Index

Sr.No.	Title of Chapter	Page No.
01	Abstract	5
02	Introduction	6
03	Methodology	8
04	Implementation and Results	10
05	System Requirements	14
06	Conclusion	14
07	References	16

ABSTRACT

Web browsers silently record every website a user visits, every search they make and every file they download. In a digital forensic investigation this browsing history is one of the most valuable sources of evidence about a suspect's activities, interests and timeline. Modern Chromium-based browsers such as Google Chrome, Microsoft Edge and Brave store this history in a single SQLite database file named `History`. The file is locked while the browser is running, its timestamps are stored as microseconds since 1 January 1601 and the data is spread over several related tables, so it cannot be read directly or safely by an investigator.

This project presents the design and development of a Browser History Forensic Analysis Tool that automates the standard forensic workflow of acquire, preserve, analyse and report. The tool copies the History database to a working folder, computes a SHA-256 hash of the evidence so that its integrity can be proved later, reads the `urls`, `visits` and `downloads` tables, converts the WebKit timestamps into readable local date and time, and reconstructs a chronological timeline of the user's browsing activity. It also reports the most visited domains, daily activity, downloaded files and keyword search hits, and exports everything as CSV files and a self-contained HTML report that can be attached to a case file.

The tool is written in about 280 lines of Python using only the standard library, so it runs on Windows, Linux and macOS without installing anything extra. It was tested with a synthetic sample database and with the live Microsoft Edge history of a Windows 11 computer, and seven automated test cases verify every function. The result is a small, transparent and easily explainable forensic utility that demonstrates evidence handling, artifact parsing and timeline analysis.

INTRODUCTION

In today's world almost every activity passes through a web browser: searching for information, reading e-mail, shopping, banking, social media and downloading files. Because of this, the browser has become one of the richest sources of digital evidence. When a computer is examined in a cyber-crime case, an insider-threat investigation or a corporate incident, the first questions investigators ask are "which websites did this person visit, when, how often, and what did they download?" The answers are stored by the browser itself in its history database.

Digital forensics is the application of scientific methods to collect, preserve, analyse and present digital evidence in a way that is acceptable in a court of law. The National Institute of Standards and Technology (NIST) describes the forensic process in four phases: collection, examination, analysis and reporting. Two principles are central to every phase. First, the original evidence must never be modified; the examiner works on a verified copy. Second, the integrity of the evidence must be provable, which is done with cryptographic hash functions such as SHA-256: if the hash of the copy equals the hash of the original, the copy is identical bit for bit.

Browser artifacts. Chromium-based browsers (Google Chrome, Microsoft Edge, Brave, Opera) keep each user profile in a folder such as `C:\Users\<name>\AppData\Local\Google\Chrome\User Data\Default`. Inside it, the file `History` is a SQLite database with the following important tables:

- `urls` – one row per unique URL with its title, visit count and typed count;
- `visits` – one row per visit with the time of the visit and a reference to the URL;
- `downloads` – one row per downloaded file with the saved path, source URL, start time and size.

Times are stored in the WebKit format: a 64-bit number of microseconds since 1 January 1601 UTC. To obtain the familiar Unix time (seconds since 1970) the value is divided by one million and 11 644 473 600 seconds are subtracted.

Problem. Commercial forensic suites such as EnCase, FTK and Magnet AXIOM can parse browser history but they are expensive and work as black boxes. Opening the History file manually in a SQLite viewer is possible but slow, it offers no hashing, the timestamps remain unreadable and there is no report. Moreover, the browser keeps the file locked, so a careless examiner may fail to open it or may even alter it.

Objectives of the project.

1. To understand how Chromium browsers store history and downloads.
2. To acquire the History file without altering the original and to prove its integrity using SHA-256.

3. To convert WebKit timestamps into readable local time and reconstruct a chronological timeline.
4. To provide useful analysis for an investigator: top visited domains, daily activity, keyword search and download listing.
5. To generate CSV and HTML reports that can be attached to a case file.
6. To keep the tool small, portable and easy to explain.

Scope. The tool supports Chromium-family browsers on Windows, Linux and macOS. It does not parse Firefox (which uses a different file, `places.sqlite`), cookies, cache or saved passwords, and it does not recover deleted records. These are left as future work.

METHODOLOGY

The tool follows the four-phase forensic process recommended by NIST. Each phase is implemented as a small group of Python functions in a single file, `bhft.py`.

1. Acquisition (collection)

The examiner gives either the name of a browser (`-browser edge`) or the path of a History file taken from a seized machine (`-db path`). The tool copies the file into an output folder with a byte-for-byte copy. A byte copy is possible even while the browser holds the SQLite lock. It then computes the SHA-256 hash of the original and of the copy; if the two differ the tool aborts, because the integrity of the evidence cannot be guaranteed. The hash is printed and written into the report so that anyone can later verify that the evidence was not altered.

2. Examination (parsing)

The copy is opened with Python's built-in `sqlite3` module in read-only mode (URI `file:...?mode=ro`). One SQL query joins the `visits` and `urls` tables and orders the rows by visit time:

```
SELECT v.visit_time, u.url, u.title, u.visit_count, u.typed_count
FROM visits v JOIN urls u ON v.url = u.id
ORDER BY v.visit_time
```

Each WebKit timestamp is converted to local time with the formula

$$t_{unix} = \frac{\mu}{10^6} - 11\,644\,473\,600$$

where μ is the stored microsecond value. The `downloads` table is read in the same way. Every visit becomes a record with time, URL, title, domain, visit count and typed count.

3. Analysis

Four simple analyses are performed on the list of visit records:

- **Timeline**: the records already sorted by time form the activity timeline; the first and last visit give the activity window.
- **Top domains**: the domain of each URL is counted and the n most frequent are listed.
- **Daily activity**: visits are counted per calendar date to show on which days the user was active.
- **Keyword search**: a case-insensitive search in URLs and page titles (for example "bank" or "invoice") lists the visits of interest to the investigator.

4. Reporting

The timeline and the downloads are written to `timeline.csv` and `downloads.csv` so they can be opened in Excel. A self-contained HTML report (`report.html`) is generated with the evidence details (source file, SHA-256, acquisition time, examiner machine), summary statistics, top domains, daily activity, keyword hits, downloads and the full timeline.

Architecture of the tool

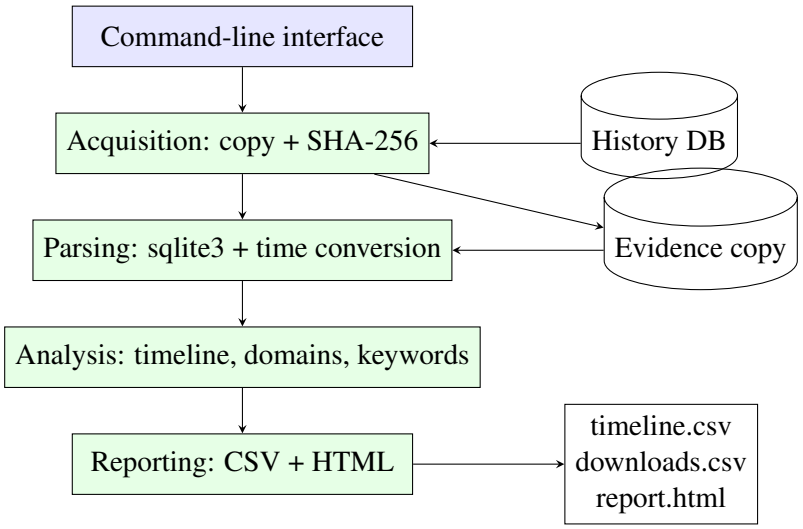


Figure 1: Architecture of the Browser History Forensic Analysis Tool

Algorithm

1. Read command-line arguments (browser or file path, output folder, keyword, number of top domains).
2. If the History file does not exist, stop with an error.
3. Copy the file to the output folder; compute SHA-256 of source and copy; stop if they differ.
4. Open the copy read-only; run the join query; convert each timestamp; build the visit list.
5. Read the downloads table.
6. Count domains and daily visits; filter visits by keyword.
7. Write CSV files and the HTML report; print the summary on the console.

IMPLEMENTATION AND RESULTS

The project consists of three Python files, available at `https://github.com/VishwajeetTulse/browser-history-forensic-tool`:

- `bhft.py` – the tool itself (about 280 lines, standard library only);
- `make_sample_db.py` – creates a realistic sample History database for safe demonstration;
- `test_bhft.py` – seven automated tests (pytest) covering timestamp conversion, hashing, parsing, keyword search, domain ranking and an end-to-end run.

The key function that converts Chromium timestamps:

```
WEBKIT_EPOCH_DELTA = 11644473600   # seconds between 1601-01-01 and 1970-01-01

def webkit_to_datetime(micros):
    if not micros:
        return None
    seconds = micros / 1_000_000 - WEBKIT_EPOCH_DELTA
    return dt.datetime.fromtimestamp(seconds, dt.timezone.utc).astimezone()
```

Running the tool

```
python make_sample_db.py                                    # create demo database
python bhft.py --db samples/History --out output --keyword bank
python bhft.py --browser edge                               # live Edge profile
```

Console output

```
======================================================================
 Browser History Forensic Analysis Tool
======================================================================
[1] Acquiring evidence from: samples/History
    SHA-256 : 94f700d91be6a7f4e05f483ac4db8267bac466fbb7d7e25ccf56e52cbefdf9a2
    Working copy saved to: output\History_evidence_copy.db
[2] Parsing history database ...
    56 visits, 3 downloads found
[3] Analysing ...

    Top 10 domains:
          7  docs.python.org
          7  www.wikipedia.org
          7  www.youtube.com
          6  netbanking.examplebank.com
```

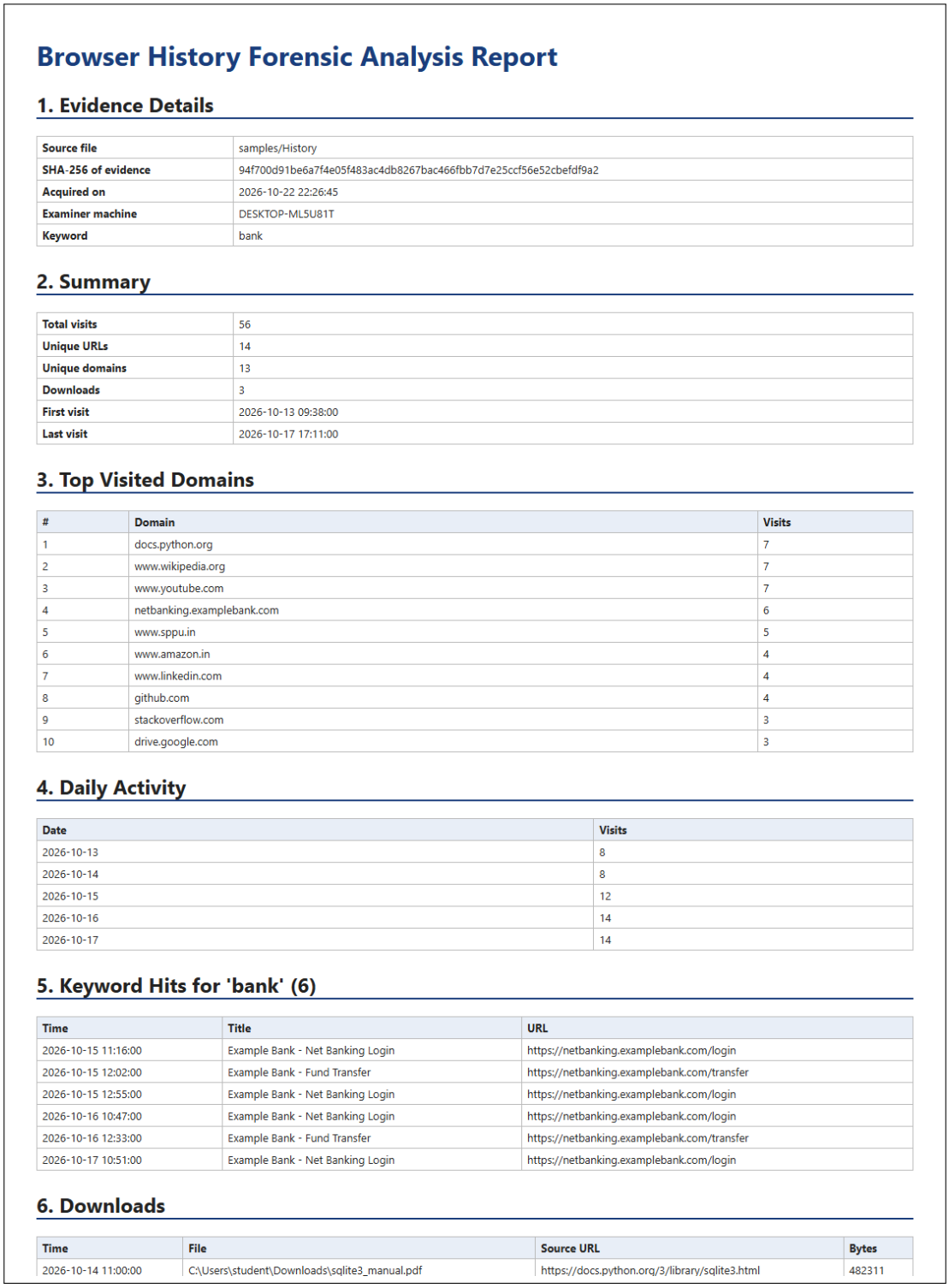
```
          5  www.sppu.in
          4  www.amazon.in
          4  www.linkedin.com
          4  github.com
          3  stackoverflow.com
          3  drive.google.com

    Activity window: 2026-10-13 09:38:00  ->  2026-10-17 17:11:00

    Keyword 'bank': 6 hit(s)
      2026-10-15 11:16:00  https://netbanking.examplebank.com/login
      2026-10-15 12:02:00  https://netbanking.examplebank.com/transfer
      2026-10-15 12:55:00  https://netbanking.examplebank.com/login
      2026-10-16 10:47:00  https://netbanking.examplebank.com/login
      2026-10-16 12:33:00  https://netbanking.examplebank.com/transfer
      2026-10-17 10:51:00  https://netbanking.examplebank.com/login

[4] Writing reports ...
    timeline.csv, downloads.csv and report.html written to 'output'
Done.
```

Generated HTML report



Browser History Forensic Analysis Report

1. Evidence Details

Source file	samples/History
SHA-256 of evidence	94f700d91be6a7f4e05f483ac4db8267bac466fbb7d7e25ccf56e52cbefdf9a2
Acquired on	2026-10-22 22:26:45
Examiner machine	DESKTOP-ML5U81T
Keyword	bank

2. Summary

Total visits	56
Unique URLs	14
Unique domains	13
Downloads	3
First visit	2026-10-13 09:38:00
Last visit	2026-10-17 17:11:00

3. Top Visited Domains

#	Domain	Visits
1	docs.python.org	7
2	www.wikipedia.org	7
3	www.youtube.com	7
4	netbanking.examplebank.com	6
5	www.sppu.in	5
6	www.amazon.in	4
7	www.linkedin.com	4
8	github.com	4
9	stackoverflow.com	3
10	drive.google.com	3

4. Daily Activity

Date	Visits
2026-10-13	8
2026-10-14	8
2026-10-15	12
2026-10-16	14
2026-10-17	14

5. Keyword Hits for 'bank' (6)

Time	Title	URL
2026-10-15 11:16:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-15 12:02:00	Example Bank - Fund Transfer	https://netbanking.examplebank.com/transfer
2026-10-15 12:55:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-16 10:47:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login
2026-10-16 12:33:00	Example Bank - Fund Transfer	https://netbanking.examplebank.com/transfer
2026-10-17 10:51:00	Example Bank - Net Banking Login	https://netbanking.examplebank.com/login

6. Downloads

Time	File	Source URL	Bytes
2026-10-14 11:00:00	C:\Users\student\Downloads\sqlite3_manual.pdf	https://docs.python.org/3/library/sqlite3.html	482311

Figure 2: HTML forensic report produced by the tool (sample database, keyword “bank”)

Testing

All seven automated tests pass:

```
test_bhft.py::test_webkit_epoch_conversion PASSED
test_bhft.py::test_sha256_matches_hashlib PASSED
test_bhft.py::test_read_visits_sorted_and_complete PASSED
test_bhft.py::test_downloads_read PASSED
test_bhft.py::test_keyword_search_case_insensitive PASSED
test_bhft.py::test_top_domains_order PASSED
test_bhft.py::test_end_to_end_cli PASSED
============================== 7 passed in 0.21s ===============================
```

The tool was also run on the live Microsoft Edge profile of a Windows 11 laptop. It copied the locked History file, printed its SHA-256 hash and produced the timeline, confirming that the default profile paths and the acquisition step work on real data.

SYSTEM REQUIREMENTS

The tool was developed on a Windows 11 laptop but, because it uses only the Python standard library, it runs on any computer that has Python 3. Below is the minimum system requirement for its use.

Hardware Requirement

- Personal computer with a 1 GHz or faster processor
- 2 GB RAM
- 500 MB free disk space for the evidence copy and reports

Software Requirement

- Operating system: Windows 10/11, Linux or macOS
- Python 3.9 or newer (no third-party packages are needed to run the tool)
- pytest (optional, only to run the automated tests)
- Any web browser to view the HTML report; Microsoft Excel or LibreOffice to open the CSV files
- Visual Studio Code or any text editor for development

CONCLUSION

In this project a small but complete digital forensic tool for browser history has been designed and implemented. The tool acquires the Chromium History database without touching the original file, proves the integrity of the evidence with a SHA-256 hash, decodes the WebKit timestamps that make the raw data unreadable, and reconstructs a chronological timeline of the user's browsing activity. It summarises the most visited domains, daily activity, downloads and keyword hits, and produces CSV and HTML reports that an investigator can attach directly to a case file.

The implementation is a single Python file with no external dependencies, which makes it portable, transparent and easy to explain. Seven automated tests verify every function, and the tool was validated on both a synthetic database and a real Microsoft Edge profile.

The tool will be useful to forensic examiners, incident-response teams, students of digital forensics and anyone who needs to understand browser evidence. In future it can be extended to

Firefox and Safari, to cookies, cache and form data, to recovery of deleted records from SQLite free pages, and to a graphical user interface.

REFERENCES

1. J. Oh, S. Lee and S. Lee, "Advanced evidence collection and analysis of web browser activity," *Digital Investigation*, vol. 8, pp. S62–S70, 2011.
2. D. J. Ohana and N. Shashidhar, "Do private and portable web browsers leave incriminating evidence?," *EURASIP Journal on Information Security*, 2013.
3. D. Rathod, "Web browser forensics: Google Chrome," *International Journal of Advanced Research in Computer Science*, vol. 8, no. 7, 2017.
4. K. Kent, S. Chevalier, T. Grance and H. Dang, *Guide to Integrating Forensic Techniques into Incident Response*, NIST Special Publication 800-86, 2006.
5. E. Casey, *Digital Evidence and Computer Crime*, 3rd ed., Academic Press, 2011.
6. The Chromium Projects, "User Data Directory," `https://www.chromium.org/developers/design-documents/user-data-dir/`
7. SQLite Consortium, "SQLite Documentation," `https://www.sqlite.org/docs.html`
8. Python Software Foundation, "sqlite3, hashlib, datetime modules," `https://docs.python.org/3/library/`